services\orchestrator.py

```
import os
import json
from typing import Dict, List, Any
from datetime import datetime, timezone, timedelta
from services.analysis.trend_analyzer import get_cve_trends_last_30_days
from services.analysis.severity_analyzer import get_severity_card_counts,
get_severity_percentages
from services.analysis.cwe_processor import get_vendor_risk_analysis,
get_weighted_cwe_analysis
from services.data.api_client import api_client
from services.cache.cache_manager import cache_manager
from services.data.data_source_config import data_source_config
import config

class DataOrchestrator:
"""Clean orchestrator using separate visualization modules with caching
and dynamic data sources"""

def __init__(self):
data_source_config.log_data_source_status()
self._vendor_risk_cache = None
self._vendor_risk_cache_time = None

def get_dashboard_data(self, year=None, month=None, day=None,
severity_filter=None, timeline_years=1, load_historical=False):
"""Get all dashboard data with separate data sources for different
components"""
print(f"[Orchestrator] Loading dashboard data with filters: year={year},
month={month}, day={day}, severity={severity_filter},
timeline_years={timeline_years}, load_historical={load_historical}")

try:
# 1. CVE Trends (Last 30 days)
print("[Orchestrator] Getting CVE trends (last 30 days) - unfiltered")
cve_trends = self._get_always_30_day_trends()

# 2. Vulnerabilities Over Time
if load_historical and timeline_years > 0:
print(f"[Orchestrator] Getting vulnerabilities over time (last
{timeline_years} years) - HISTORICAL LOAD")
vulnerabilities_timeline = self._get_timeline(timeline_years)
else:
print("[Orchestrator] Skipping historical timeline (not requested)")
vulnerabilities_timeline = {'labels': [], 'values': [], 'total_cves': 0,
'months_covered': 0, 'raw_data': {}}

# 3. Filtered data for Cards and Pie Chart ONLY
print("[Orchestrator] Getting filtered data for cards and pie chart")
filtered_cves = self.get_filtered_cves(year=year, month=month, day=day)

if severity_filter:
from utils.filters import FilterService
filter_service = FilterService()
filtered_cves = filter_service.filter_by_severity(filtered_cves,
severity_filter)

# 4. Calculate severity counts
from collections import Counter
severity_counts = Counter()
for cve in filtered_cves:
```

```
severity = cve.get('Severity', 'UNKNOWN').upper()
if severity in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW']:
severity_counts[severity] += 1
else:
severity_counts['UNKNOWN'] += 1

severity_metrics = {
'CRITICAL': severity_counts.get('CRITICAL', 0),
'HIGH': severity_counts.get('HIGH', 0),
'MEDIUM': severity_counts.get('MEDIUM', 0),
'LOW': severity_counts.get('LOW', 0),
'UNKNOWN': severity_counts.get('UNKNOWN', 0),
'total_cves': len(filtered_cves)
}

total = len(filtered_cves)
if total > 0:
severity_percentages = {}
for key in ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW', 'UNKNOWN']:
percentage = round((severity_counts.get(key, 0) * 100 / total))
severity_percentages[key] = f"{percentage}%"
else:
severity_percentages = {k: "0%" for k in ['CRITICAL', 'HIGH', 'MEDIUM',
'LOW', 'UNKNOWN']}

# 5. Vendor Risk Analysis - Cache for 24 hours
cache_expired = False
if self._vendor_risk_cache_time:
cache_age = datetime.now() - self._vendor_risk_cache_time
if cache_age > timedelta(hours=24):
cache_expired = True
print("[Orchestrator] Vendor risk cache expired (>24 hours old)")

if self._vendor_risk_cache and not cache_expired:
cache_age_str = ""
if self._vendor_risk_cache_time:
age = datetime.now() - self._vendor_risk_cache_time
hours = int(age.total_seconds() / 3600)
cache_age_str = f" (cached {hours}h ago)"
print(f"[Orchestrator] Using cached vendor risk analysis{cache_age_str}")
vendor_risk, weighted_vendor_risk = self._vendor_risk_cache
else:
print("[Orchestrator] Loading vendor risk analysis for 2025 (will cache
for 24 hours)")
try:
vendor_risk = get_vendor_risk_analysis()
weighted_vendor_risk = get_weighted_cwe_analysis()
self._vendor_risk_cache = (vendor_risk, weighted_vendor_risk)
self._vendor_risk_cache_time = datetime.now()
print("[Orchestrator] Vendor risk cached (expires in 24 hours)")
except Exception as e:
print(f"[Orchestrator] Error loading vendor risk analysis: {e}")
vendor_risk = self._get_empty_vendor_risk()
weighted_vendor_risk = {'indices': [], 'labels': [], 'values': []}

cutoff_info, explanation = data_source_config.get_data_source_cutoff()
if year and month and day:
note_text = f"Cards & Pie Chart: {year}-{month:02d}-{day:02d} | Trends:
Last 30 days"
elif year and month:
note_text = f"Cards & Pie Chart: {year}-{month:02d} | Trends: Last 30
days"
elif year:
note_text = f"Cards & Pie Chart: {year} data | Trends: Last 30 days"
```

```
elif severity_filter:
note_text = f"Cards & Pie Chart: {severity_filter.lower()} severity |
Trends: Last 30 days"
else:
note_text = f"Cards & Pie Chart: Last 30 days | Trends: Last 30 days"
if load_historical:
note_text += f" | Timeline: Last {timeline_years} year(s)"

print(f"[Orchestrator] Data loaded successfully:")
print(f"  - Filtered CVEs (cards/pie): {len(filtered_cves)}")
print(f"  - CVE Trends data points: {len(cve_trends.get('labels', []))}")
print(f"  - Timeline data points:
{len(vulnerabilities_timeline.get('labels', []))}")
print(f"  - Severity counts: C={severity_metrics['CRITICAL']},
H={severity_metrics['HIGH']}, M={severity_metrics['MEDIUM']},
L={severity_metrics['LOW']}")

dashboard_data = {
'metrics': severity_metrics,
'total_cves': len(filtered_cves),
'severity_percentage': severity_percentages,
'timeline_data_days': cve_trends,
'timeline_data_years': vulnerabilities_timeline,
'cwe_radar': vendor_risk.get('top_10_cwes', {'indices': [], 'labels': [],
'values': []}),
'cwe_radar_all': vendor_risk,
'cwe_radar_weighted': weighted_vendor_risk,
'cwe_radar_descriptions': self._get_cwe_descriptions(),
'latest_cves': filtered_cves,
'available_years': list(range(datetime.now().year, 2009, -1)),
'available_months': list(range(1, 13)),
'note_text': note_text,
'historical_loaded': load_historical
}

print(f"[Orchestrator] Dashboard data structure created successfully")
return dashboard_data

except Exception as e:
print(f"[Orchestrator] Error in get_dashboard_data: {e}")
import traceback
traceback.print_exc()
raise e

def _get_empty_vendor_risk(self):
return {
'top_5_cwes': {'indices': [], 'labels': [], 'values': []},
'top_10_cwes': {'indices': [], 'labels': [], 'values': []},
'all_cwes': {'indices': [], 'labels': [], 'values': []},
'severity_matrix': {},
'cwe_30_day_counts': {},
'total_cves_analyzed': 0,
'years_analyzed': []
}

def _get_always_30_day_trends(self) -> Dict[str, Any]:
try:
return get_cve_trends_last_30_days()
except Exception as e:
print(f"[Orchestrator] Error getting 30-day trends: {e}")
return {'labels': [], 'values': [], 'total_cves': 0, 'date_range':
{'start': '', 'end': ''}}

def _get_timeline(self, years=1) -> Dict[str, Any]:
```

```
try:
from services.analysis.trend_analyzer import
get_vulnerabilities_over_time_last_n_years
return get_vulnerabilities_over_time_last_n_years(years)
except Exception as e:
print(f"[Orchestrator] Error getting timeline: {e}")
return {'labels': [], 'values': [], 'total_cves': 0, 'months_covered': 0,
'raw_data': {}}

def get_filtered_cves(self, year=None, month=None, day=None) ->
List[Dict[str, Any]]:
print(f"[Orchestrator] Getting filtered CVEs: year={year}, month={month},
day={day}")
cutoff_info, explanation = data_source_config.get_data_source_cutoff()

if not year:
print(f"[Orchestrator] Using API data for recent data")
cves = api_client.get_cves_last_30_days()
print(f"[Orchestrator] API returned {len(cves)} CVEs")
return cves

if data_source_config.should_use_historical(year, month):
print(f"[Orchestrator] Using HISTORICAL data for {year}-{month or 'all'}")
from services.analysis.trend_analyzer import get_filtered_historical_cves
return get_filtered_historical_cves(year=year, month=month, day=day)
else:
print(f"[Orchestrator] Using API data for {year}-{month or 'all'}")
# Get all CVEs from last 30 days and filter them manually
cves = api_client.get_cves_last_30_days()

# Filter the CVEs by date
filtered_cves = []
for cve in cves:
pub_date = cve.get('Published', '')
if pub_date:
try:
if 'T' in pub_date:
dt = datetime.fromisoformat(pub_date.replace('Z', '+00:00'))
else:
dt = datetime.strptime(pub_date[:10], '%Y-%m-%d')

# Match year
if int(year) == dt.year:
# Match month if provided
if month is None or int(month) == dt.month:
# Match day if provided
if day is None or int(day) == dt.day:
filtered_cves.append(cve)
except:
continue

print(f"[Orchestrator] Filtered API data: {len(filtered_cves)} CVEs match
{year}-{month or 'all'}-{day or 'all'}")
return filtered_cves

def get_cve_detail(self, cve_id: str) -> Dict[str, Any]:
return api_client.get_cve_detail(cve_id)

def clear_cache(self):
api_client.clear_cache()
cache_manager.clear_all()
self._vendor_risk_cache = None
self._vendor_risk_cache_time = None
```

```
def get_data_source_status(self) -> Dict[str, Any]:
return data_source_config.get_status()

def _get_cwe_descriptions(self) -> Dict[str, str]:
return {}

data_orchestrator = DataOrchestrator()
```